

Pinning “Reflection” on the Agenda: Investigating Reflection in Human–LLM Co-Creation for Creative Coding

Anqi Wang

Hong Kong University of Science and
Technology
Hong Kong, Hong Kong
awangan@connect.ust.hk

Zhizhuo Yin

Hong Kong University of Science and
Technology (Guangzhou)
Guangzhou, China
zyin190@connect.hkust-gz.edu.cn

Yulu Hu

Hong Kong University of Science and
Technology (Guangzhou)
Guangzhou, China
yhu635@connect.hkust-gz.edu.cn

Yuanyuan Mao

Hong Kong University of Science and
Technology (Guangzhou)
Guangzhou, China
ymao709@connect.hkust-gz.edu.cn

Lei Han

Hong Kong University of Science and
Technology (Guangzhou)
Guangzhou, China
lhan229@connect.hkust-gz.edu.cn

Xin Tong

Hong Kong University of Science and
Technology (Guangzhou)
Guangzhou, China
xint@hkust-gz.edu.cn

Keqing Jiao

Carnegie Mellon University
Pittsburgh, Pennsylvania, USA
zixiong@andrew.cmu.edu

Pan Hui*

Hong Kong University of Science and
Technology (Guangzhou)
Guangzhou, China
panhui@hkust-gz.edu.cn

Abstract

Large language models (LLMs) are increasingly integrated into creative coding, yet how users reflect, and how different co-creation conditions influence reflective behavior, remains underexplored. This study investigates situated, moment-to-moment reflection in creative coding under two prompting strategies: the entire task invocation (T1) and decomposed subtask invocation (T2), to examine their effects on reflective behavior. Our mixed-method results reveal three distinct reflection types and show that T2 encourages more frequent, strategic, and generative reflection, fostering diagnostic reasoning and goal redefinition. These findings offer insights into how LLM-based tools foster deeper creative engagement through structured, behaviorally grounded reflection support.

CCS Concepts

• **Applied computing** → **Media arts**; • **Human-centered computing** → Human computer interaction (HCI); **Empirical studies in HCI**.

Keywords

Human-AI collaboration, LLMs, reflection, creative coding, design process

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
CSCW Companion '25, Bergen, Norway
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1480-1/25/10
<https://doi.org/10.1145/3715070.3749234>

ACM Reference Format:

Anqi Wang, Zhizhuo Yin, Yulu Hu, Yuanyuan Mao, Lei Han, Xin Tong, Keqing Jiao, and Pan Hui. 2025. Pinning “Reflection” on the Agenda: Investigating Reflection in Human–LLM Co-Creation for Creative Coding. In *Companion of the Computer-Supported Cooperative Work and Social Computing (CSCW Companion '25)*, October 18–22, 2025, Bergen, Norway. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3715070.3749234>

1 Introduction

Recent advances in large language models (LLMs), such as Code Llama and ChatGPT-4, which demonstrated significant capabilities in interpreting natural language descriptions of programming tasks [2, 23, 29] and generating functional, contextually appropriate code [8, 9, 11, 20, 33], has been applied to broad art and design domains [43]. These developments have enabled a novel form of **human–AI co-creation in creative coding** (e.g., [1])—one that combines LLMs’ technical fluency with the iterative, exploratory nature of artistic practice. Creative coding [34] emphasizes artistic concept and visual appeal through iterative codes, yet it presents unique challenges: artists must both imagine system-level behaviors and operationalize them through complex programming environments like P5.js [30]. **Reflection** is central to navigating this creative complexity, enabling artists to evaluate goals, reframe strategies, and evolve thinking [7, 18, 21]. Moreover, it fosters self-awareness, allowing users to adapt thinking and overcome obstacles, optimizing creative solutions. In human-LLM co-creation, reflection takes on new dynamics: users respond not only to their work but also to machine-generated suggestions, bugs, or alternatives. This shifts reflection from purely internal evaluation to an interactive process shaped by dialogic rhythm and prompt framing. The mode of co-creation—whether through whole-program generation or decomposed subtasks—can influence when and how reflection occurs, and whether it leads to surface-level fixes or deeper conceptual reframing.

However, existing research largely overlooks how reflection unfolds within human-LLM co-creation, focusing instead on technical performance or post-hoc interpretation. To our knowledge, only Ford et al. [19] has investigated AI-supported reflection in the context of music composition. What remains unclear is how different forms of reflection emerge in real-time during co-creative dialogue with LLMs, and how prompt framing might shape that process. To address these issues, we investigate reflection as a situated, behavioral phenomenon during creative coding with LLMs under different co-creation approaches. Our study asks: *How do artists reflect during creative coding with LLMs, and how do different types of reflection shape the human-LLM co-creation process under varying co-creation conditions?*

We conducted our study in two co-creation ways with LLMs involving art practitioners (N=22) with diverse programming backgrounds, one invoking the entire program (T1), the other subtasks solving a problem (T2). Through qualitative coding and frequency analysis, we found that the T2 condition increased reflection frequency and shifted it from reactive repair to proactive goal reframing and strategic reasoning, highlighting the role of such strategy in fostering deeper reflection. This contributes: (1) how Baumer’s reflection typology applies and adapts in the context of LLM-based creative coding, revealing distinct patterns—in the moment-to-moment flow; (2) Empirical evidence showing how task-decomposed prompting fosters deeper, more strategic reflection; (3) Design implications for rethinking reflective prompting beyond productivity scaffolding, but as a catalyst for creative reasoning and adaptive goal-setting.

2 Related Works

Recent advances in Large Language Models (LLMs) like ChatGPT-4 and Code Llama have enabled them to understand coding tasks from natural language prompts [2, 23, 29] and produce coherent, contextually appropriate code [8, 9, 11, 20, 33]. These breakthroughs have spurred interest in using LLMs as co-creators in creative tasks (e.g., [1, 6, 10, 15, 41]), including tools that support creative coding. For example, Angert et al. [1] developed an environment enabling artists to generate and refine generative art through interaction and direct code edits. However, much of this work emphasizes boosting productivity via LLMs, often neglecting the artist’s reflective role, potentially leading to over-reliance and diminished creative agency in the final outcome.

Reflection is a foundational concept in HCI [38], particularly in creative domains like writing, design, and project documentation [12, 13, 16, 22, 24, 25, 32, 39, 40, 45]. It includes deliberate thought for problem understanding [37] and the re-evaluation of past experience [3]. Baumer et al. offered a comprehensive reflection framework [4, 5], identifying three components: **Breakdown**: disruptions that prompt reevaluation; **Inquiry**: data collection and conceptual exploration; **Transformation**: shifts in understanding or creative direction. Despite its relevance, empirical work on reflection in LLM-assisted creativity remains scarce, motivating our study of how reflection unfolds in creative coding with LLMs.

3 Methods

To investigate reflection in human-LLM co-creation on creative tasks, we conducted a mixed-method study with controlled experiments and qualitative interviews. Building on prior design guidelines for open-ended objectives [19], the study received ethics approval, with all participants providing signed consent and voluntarily participating without compensation. We used the CodeLlama-Instruct-34B model [36] for its strong code generation and natural language capabilities. As Meta released only model weights, we built a custom ChatGPT-style frontend (Figure 1) and deployed the model on an Ubuntu server with 6×NVIDIA A800 80GB GPUs.

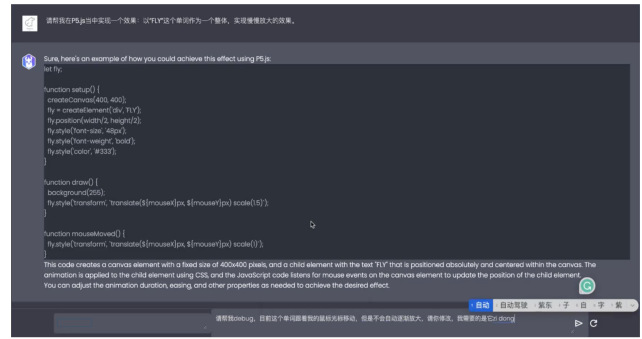


Figure 1: The user interface we developed for Code-Llama.

Procedure. Through e-application forms collecting demographic information, posted online in social media groups and offline campus in mainland of China, we screened and recruited a total of 22 participants attended our study with balanced backgrounds (see Table 1). Our participants are artist- and design-researchers who have wide creative coding needs in their daily creation. We conducted

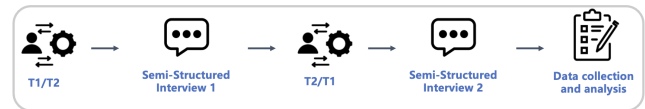


Figure 2: Overall study procedure

face-to-face studies in a campus laboratory, with each participant attending individually. Following prior studies on human-LLM co-creation paradigms [10, 27, 35], we conducted a within-subjects design where each participant completed both conditions (T1 and T2)-two co-creation ways [10, 27, 35]: **Condition T1**-invoking Llama with the entire coding goal and **Condition T2**-decomposing the programming task into subtasks and sequentially invoking the LLM to address each, by collecting observation, think-aloud, and semi-structured interview. The order of conditions was counter-balanced across participants to control for sequence effects. In both two conditions, we asked participants to create a creative coding in the same, self-defined themes but in different forms using P5.js² web-end under two conditions. Required guidelines includes 1)

²P5.js is a JavaScript library designed for creative coding on web browsers, facilitating the creation of interactive visuals through code

Table 1: Summary of democratized information of participants in the user study. Among the 22 participants, 10 were male and 12 were female, aged from 22 to 32. Programming experience is categorized into five levels, from low to high: Level 1 - Little experience; Level 2 - Some experience; Level 3 - Moderate experience; Level 4 - Substantial experience; Level 5 - Professional. P5.js or Processing experience is categorized into four levels, from low to high: Level 1 - Little experience; Level 2 - Some experience; Level 3 - Moderate experience; Level 4 - Substantial experience.

NO.	Gender	Age	Profession	Programming Experience	Programming Language	Other Creative Programming Tool	Used P5.js/Processing
P16	Male	26	VR Artist	5	C++, Java, Python, Matlab	None	4
P19	Female	28	Computer Artist & Robot Arm Artist	4	Java	TouchDesigner, Arduino, Grasshopper	4
P20	Female	24	Animation Artist	4	Java	None	3
P1	Female	24	Visual Communication Designer	3	Java	Arduino	3
P2	Male	32	Computer Artist & Generative AI Artist	3	HTML	TouchDesigner, maxmsp	3
P8	Female	23	Industrial Designer	3	C#	None	3
P9	Male	26	Computer Artist	3	Java	Others	3
P11	Male	22	Film Artist	3	C#, Python	None	1
P14	Male	27	Computational Creative Artist	3	C#, Java, Python	N/A	3
P17	Male	28	Architect	3	Java, Python	Grasshopper	3
P3	Male	26	Environmental Artist & Computational Creative Artist	2	C#	Arduino, Grasshopper, Houdini	2
P7	Female	22	Interaction Designer	2	C, Python	None	2
P10	Female	24	Industrial Designer	2	C#	Arduino	1
P12	Female	22	Digital Media Artist	2	Java	None	3
P15	Male	27	Interactive Image Artist	2	Java	TouchDesigner, Arduino	2
P18	Male	22	Interaction Designer	2	C#	Others	1
P4	Female	24	Product Designer	1	Java	Others	1
P5	Female	26	Architect	1	Python	Arduino	1
P6	Female	24	Visual Communication Designer	1	N/A	Others	1
P13	Male	25	Landscape Artist	1	C#	Others	1
P21	Female	29	Industrial Designer	1	Python	Arduino	1
P22	Female	31	Environmental Artist	1	C#	Others	1

1

with a minimum two visual elements 2) in interactive and 3) visual-appeal way in each task. Before each condition, we explained the prompting strategy with examples, and verified understanding via a brief quiz. We analyzed possible sequence effects by comparing reflection frequency and type between orders and found no significant order effect. The study procedure is shown in the Figure 2.

Data Analysis. To analyze reflection behaviors, we employed a mixed-method approach combining qualitative and quantitative analysis. We began by conducting a theory-informed thematic analysis, drawing on Baumer’s reflection framework [4, 5] to identify three primary reflection types: Breakdown, Inquiry, and Transformation. These categories were refined through an iterative process during data collection, and reflection was defined as any user behavior involving rethinking, questioning, or revising ideas in response to LLM feedback. Both deductive and inductive strategies were applied: the deductive approach mapped behaviors to Baumer’s framework, while the inductive approach captured emergent behaviors beyond the system, such as independent code editing, logic reformulation, or drawing on prior experience. We segmented the data into reflection episodes, defined as a logic unit comprising a user prompt, LLM output, a reflection trigger (e.g., error or surprise), the user’s response, and the resulting course of action. Each episode was annotated with causal and conditional links (e.g., “LLM misunderstanding → user clarifies syntax”, “unexpected output → user shifts goal”) using axial coding principles [42]. A codebook was iteratively developed by two researchers, beginning with 5 transcripts coded jointly to establish shared understanding. After alignment on category definitions and coding granularity, the researchers independently coded all reflection episodes. Coding training included discussions on ambiguous cases and guided application

of Baumer’s definitions. Inter-rater reliability was assessed using Cohen’s $\kappa = 0.84$, indicating strong agreement. Disagreements were resolved through in-depth discussions until full consensus was reached. This process yielded a refined, behaviorally grounded coding scheme that supported both qualitative insights and statistical comparison.

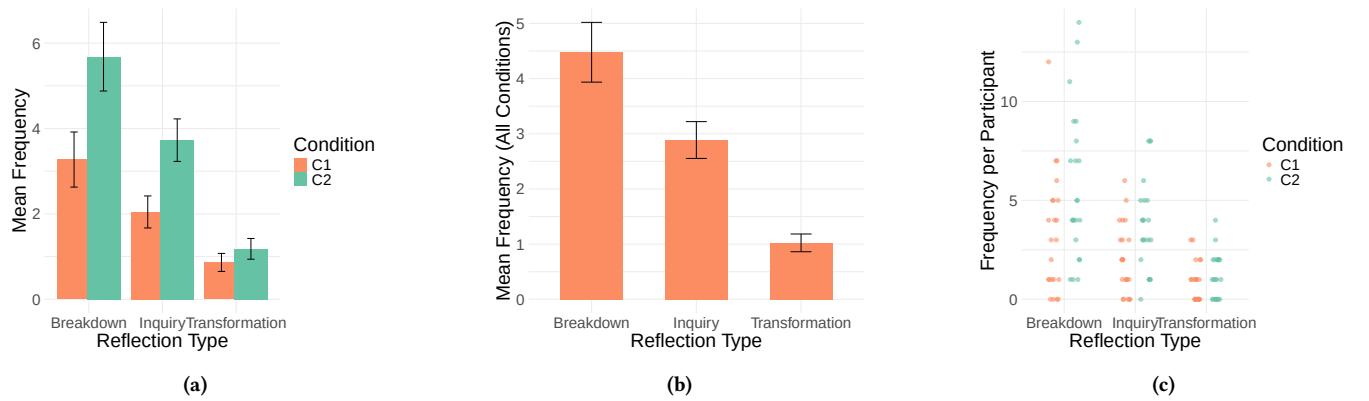
For the quantitative analysis, we analyzed the frequency of reflection instances using a non-parametric Aligned Rank Transform (ART) ANOVA [44], due to the non-normal, count-based nature of the data. The Reflection Category (Breakdown, Inquiry, Transformation) and Condition (T1, T2) were treated as fixed factors, and the reflection frequency served as the dependent variable. After assessing main and interaction effects, we conducted post-hoc pairwise comparisons and multi-factor contrasts using the ART-C procedure[17] to further examine differences between reflection types and conditions.

4 Findings

Types and Frequencies of Reflection. We identified three types of reflection based on user dialogues with LLMs and parallel behaviors in creative programming as Table 2: **Breakdown:** Triggered by errors or misunderstandings. Users refined prompts, copied errors into LLMs for debugging, or asked for explanations. Outside the LLM, they debugged independently, searched for explanations, and reused code from external sources. **Inquiry:** Emerged when users sought better outcomes or clarification. Dialogues focused on output refinement or programming help. In non-LLM behaviors, users applied prior knowledge to revise code and improve functionality. **Transformation:** Occurred when users revised their project direction due to failure or surprise. They reformulated goals

Table 2: Three reflection types characterize behaviors in: (1) interactive dialogues with LLMs; and (2) creative coding practices.

NO.	Category	Dialogues with LLMs	Other Behaviors
1	Breakdown	1) refining prompts in LLMs for precise requirement descriptions, 2) triggering debug in LLMs through copy-paste error reminders, 3) seeking explanations from LLMs for programs or codes.	1) user-initiated debugging without LLM co-creation, 2) searching and analyzing materials/documents for error explanations and debugging, 3) referencing provided sources by copying and pasting programs or codes.
2	Inquiry	1) optimizing output results based on the current program, 2) seeking information or requesting programming assistance within the design process.	1) modifying programming or codes to present results accurately based on prior knowledge.
3	Transformation	1) adjusting the initial idea or project due to unsuccessful results, 2) responding to unexpected results, 3) proposing an updated problem-solving structure to LLMs.	1) drawing inspiration or generating ideas from programs provided by external sources.

**Figure 3: (a) The frequency of reflection types at T1 and T2, with mean values and error bars. (b) The overall frequency of each reflection type. (c) Individual participants' reflection type frequencies in T1 and T2.**

in LLM interactions and adapted ideas from external inspirations outside the system. These types reflect distinct reflective strategies across the coding process, from clarifying errors to evolving creative intent.

Descriptive statistics indicated that Breakdown reflections were the most frequent (total $N = 203$), followed by Inquiry ($N = 123$), and Transformation ($N = 43$). Compared two conditions, participants reflected more frequently in T2 ($N = 222$) than in T1 ($N = 147$), with a breakdown of 118 Breakdowns, 80 Inquiries, and 24 Transformations in T2, versus 85 Breakdowns, 43 Inquiries, and 19 Transformations in T1. Across conditions, **Breakdown reflections were most prevalent**, followed by Inquiry and then Transformation.

Our non-parametric Aligned Rank Transform (ART) ANOVA statistical analysis [44] showed a significant main effect of co-creation condition on reflection frequency, ($F(1, 105) = 16.02, p < .001$), highlighting the variability in reflection frequencies across different types of reflection (Breakdown, Inquiry, and Transformation)(Figure 3a). The result also confirmed a significant main effect of Reflection Category ($F(2, 120) = 13.90, p < .0001$) (Figure 3b), indicating that different reflection types occurred with significantly different frequencies overall (Figure 3b). This suggests that the type of reflection is influenced by both the co-creation condition and participants' reflection category. While the interaction between

condition and reflection category was not statistically significant ($F(2, 105) = 2.76, p = .068$), the effect size was moderate ($\eta^2 = .05$). Although statistical power was limited, the findings suggest a potentially meaningful behavioral difference in how reflection types are shaped by the cooperation condition.

To probe these patterns, we conducted post-hoc pairwise and multi-factor contrast tests following the ART-C procedure [17]. Post-hoc pairwise comparisons showed that **Breakdown** reflections occurred significantly more often than **Inquiry** ($p < .01$), and both occurred significantly more frequently than **Transformation** ($p < .001$). Further contrast tests revealed that the **T2** condition led to significantly more Breakdown reflections (T2: $\mu = 5.68$, T1: $\mu = 3.27, p < .01$) and Inquiry reflections (T2: $\mu = 3.72$, T1: $\mu = 2.04, p < .05$) than the **T1** condition. Although Transformation reflections were also more frequent in T2 ($\mu = 1.18$) than in T1 ($\mu = 0.86$), this difference did not reach statistical significance ($p = .37$). These results suggest that the more scaffolded T2 condition encouraged greater engagement in both surface-level (Breakdown) and diagnostic (Inquiry) forms of reflection, while the deeper, goal-shifting Transformation reflections remained relatively infrequent across both conditions. Figure 3c presents a jitter plot of individual participants' reflection type frequencies in T1 and T2, highlighting the distribution of Breakdown, Inquiry, and Transformation reflections.

Reflection Usages. T1: Expressive Ambiguity and Affective Struggle. In T1, reflection is often triggered by a *Breakdown* in communication between the user and the system, leading to frustration and confusion. Users struggle with ambiguous or overly complex goals, frequently expressing dissatisfaction or emotional fatigue as they attempt to work through these challenges. For example, when encountering a technical failure, users may question their own prompt clarity, as seen in the case of P7, who remarked: “*Maybe I wasn’t clear enough... I thought it would understand what I meant.*” In this stage, users tend to focus on surface-level fixes, such as rephrasing prompts or inspecting syntax, without fundamentally addressing the underlying logic or system limitations. As P11 demonstrated through repeated phrasing adjustments without questioning the task’s conceptual framework—showing minimal progress—they stated: “*I’ve rewritten this five times, and the output remains unchanged.*” When faced with persistent failure, T1 users often simplify or abandon their original goals, opting for less complex tasks in response to accumulated frustration. For instance, P8 reflected after repeated *Breakdowns*: “*This interaction is too hard—I’ll just make a static visual.*” This response illustrates a tendency to scale back ambitions rather than reframe goals to enable creative expression within the system’s constraints. Nonetheless, T1 users sometimes re-engage after resetting their goals, albeit cautiously, as reflected in P3: “*Let’s see if I can bypass the old problem this time.*” Overall, T1 reflection is characterized as emotion-driven, syntactically oriented, and prone to stagnation when confronted with system unpredictability or misalignment.

T2: Strategic Adaptation and System-Aware Reasoning. In contrast, T2 users adopt a strategic, system-aware approach to reflection, marked by *Inquiry* and iterative exploration. Following an initial *Breakdown*, they engage in diagnostic reasoning and pursue step-by-step exploration of potential solutions, refining their strategies in response to system feedback. For example, P5 adjusted a parameter to fine-tune output: “*I found that speed 0.5 is too slow, let me try 1.*” Such users emphasize precise control over variables and actively test hypotheses, treating each failure as an opportunity for further refinement. P2, for instance, identified a conflict in system behavior and proactively revised the prompt’s logic: “*Maybe the delay command conflicts here—I’ll try replacing it with frame-count.*” Rather than abandoning goals after failure, T2 users tend to reframe or *Transform* them while preserving the underlying creative intent. This is evident in P9’s reflection: “*I wanted a sense of fluidity—maybe I can use particles instead of a path.*” Such responses highlight a metacognitive shift, as users adapt strategies to better align with system affordances. T2 users also adjust strategies dynamically. For example, P6 diagnosed a missed detail and implemented a targeted correction: “*I changed the parameter but didn’t reset the system state—maybe that’s the issue.*” Moreover, they demonstrate compressed metacognition, rapidly integrating prior knowledge with real-time feedback to make agile decisions. P10 exemplified this by drawing on past experience to identify a likely canvas issue: “*I’ve seen canvas issues before—this time it’s probably the loop interfering.*” By synthesizing accumulated insights and adapting in real time, T2 users maintain creative momentum even in the face of setbacks.

5 Discussion

Our analysis revealed a consistent pattern across both quantitative and qualitative data: while Condition 2 (T2) increased the frequency of reflection, it also shifted its nature—from reactive responses in T1 to more strategic, generative forms in T2. Although the interaction effect in the ANOVA was marginally significant ($p = .068$), qualitative evidence clearly supported this trend. In T1, reflection was often reactive and triggered by breakdowns, leading users to clarify or simplify goals. In contrast, T2 encouraged more deliberate, strategy-oriented reflection—users manipulated parameters purposefully, compared alternatives, and reframed goals. These behaviors suggest that structured, decomposed prompts in T2 facilitated diagnostic reasoning and evaluative reflection, especially during ideation and transformation.

These findings extend prior research on AI-assisted creativity [19], which has primarily emphasized high-level conceptual reflection or post-hoc interpretation. Our work advances this perspective by introducing a **fine-grained, behaviorally grounded typology of reflection—Breakdown, Inquiry, and Transformation—that emerges in the moment-to-moment flow of LLM-mediated creative programming**. This shift from retrospective abstraction to situated, real-time reflection offers a new lens for understanding reflective engagement as it unfolds dynamically within human-AI co-creation.

Beyond theoretical insight, our findings carry important implications for co-creative system design. Existing prompt tools largely emphasize productivity—generating correct, complete, or efficient outputs [1, 14, 26, 28, 31, 46]. However, our study reveals a complementary role for prompts: **provoking strategic reflection and enabling shifts in user reasoning**. To support this, systems should **adopt cooperative prompting strategies that introduce mild uncertainty, unfamiliar reasoning paths, or contrasting perspectives**. Interface modes might prioritize exploration over immediate solution-finding. For HCI practitioners, this suggests moving beyond success-oriented scaffolding to also deliberately surface productive struggle—guiding users to examine the “why” and “how” of their process at moments that matter most.

6 Limitation and Conclusion

While our study reveals how reflection unfolds in situated, short-burst interactions, future work should examine whether our typology—Breakdown, Inquiry, Transformation—applies to other creative domains such as writing or design. Further, to support broader adoption, systems should adapt cooperative prompting strategies to different user profiles and dynamically surface reflection-supportive modes based on real-time interaction cues.

Despite these, this work offers a reflection-centered approach for LLM-assisted creativity in HCI. By distinguishing reflection types across two co-creation approaches, we show how LLM configurations influence reflection’s frequency and nature. These insights provide design implications for tools that support deeper cognitive engagement beyond task completion.

Acknowledgments

This work was supported in part by the National Key Research and Development Program of China under Grant 2024YFC3307602, the

Guangzhou Municipal Nansha District Science and Technology Bureau under Contract No.2022ZD01, and the Guangdong Provincial Talent Program, Grant No.2023JC10X009.

References

- [1] Tyler Angert, Miroslav Suzara, Jenny Han, Christopher Pondoc, and Hariharan Subramonyam. 2023. Spellburst: A Node-based Interface for Exploratory Creative Coding with Natural Language Prompts. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. 1–22.
- [2] Yejin Bang, Samuel Cahyawijaya, Nayeon Lee, Wenliang Dai, Dan Su, Bryan Wilie, Holy Lovenia, Ziwei Ji, Tiezheng Yu, Willy Chung, et al. 2023. A multitask, multilingual, multimodal evaluation of chatgpt on reasoning, hallucination, and interactivity. *arXiv preprint arXiv:2302.04023* (2023).
- [3] Scott Bateman, Jaime Teevan, and Ryan W White. 2012. The search dashboard: how reflection and comparison impact search behavior. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 1785–1794.
- [4] Eric P.S. Baumer. 2015. Reflective Informatics: Conceptual Dimensions for Designing Technologies of Reflection. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems*. ACM, Seoul Republic of Korea, 585–594. doi:10.1145/2702123.2702234 TLDR: This paper reviewing and engaging with conceptual and theoretical models of reflection, organized by the disciplinary and epistemological perspectives each embodies, identifies three dimensions of reflection: breakdown, inquiry, and transformation that serve as the foundation for reflective informatics..
- [5] Eric P.S. Baumer, Vera Khovanskaya, Mark Matthews, Lindsay Reynolds, Victoria Schwanda Sosik, and Geri Gay. 2014. Reviewing reflection: on the use of reflection in interactive system design. In *Proceedings of the 2014 Conference on Designing Interactive Systems (DIS '14)*. Association for Computing Machinery, New York, NY, USA, 93–102. doi:10.1145/2598510.2598598 event-place: Vancouver, BC, Canada TLDR: A review of research on systems designed to support reflection finds that most work in this area does not actually define the concept of reflection and that most evaluations do not focus on reflection per se rather but on some other outcome arguably linked to reflection..
- [6] Bhavya Bhavya, Jinjun Xiong, and Chengxiang Zhai. 2023. Cam: A large language model-based creative analogy mining framework. In *Proceedings of the ACM Web Conference 2023*. 3903–3914.
- [7] Linda Candy. 2020. *The Creative Reflective Practitioner: Research Through Making and Practice*. Routledge, Abingdon, Oxon New York, NY.
- [8] Davide Castelvocchi. 2022. Are ChatGPT and AlphaCode going to replace programmers? *Nature* (2022).
- [9] Saikat Chakraborty, Toufique Ahmed, Yangruibo Ding, Premkumar T Devanbu, and Baishakhi Ray. 2022. NatGen: generative pre-training by “naturalizing” source code. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 18–30.
- [10] Zenan Chen and Jason Chan. 2023. Large language model in creative work: The role of collaboration modality and user expertise. Available at SSRN 4575598 (2023).
- [11] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C Desmarais, and Zhen Ming Jiang. 2023. Github copilot ai pair programmer: Asset or liability? *Journal of Systems and Software* (2023), 111734.
- [12] Peter Dalsgaard and Kim Halskov. 2012. Reflective design documentation. In *Proceedings of the Designing Interactive Systems Conference (Newcastle Upon Tyne, United Kingdom) (DIS '12)*. Association for Computing Machinery, New York, NY, USA, 428–437. doi:10.1145/2317956.2318020
- [13] Peter Dalsgaard, Kim Halskov, and Steve Harrison. 2012. Supporting reflection in and on design processes. In *Proceedings of the Designing Interactive Systems Conference*. 803–804.
- [14] Paul Denny, Juho Leinonen, James Prather, Andrew Luxton-Reilly, Thezyrie Amarouche, Brett A Becker, and Brent N Reeves. 2023. Promptly: Using prompt problems to teach learners how to effectively utilize ai code generators. *arXiv preprint arXiv:2307.16364* (2023).
- [15] Giulia Di Fede, Davide Rocchesso, Steven P Dow, and Salvatore Andolina. 2022. The Idea Machine: LLM-based Expansion, Rewriting, Combination, and Suggestion of Ideas. In *Proceedings of the 14th Conference on Creativity and Cognition*. 623–627.
- [16] Jelle van Dijk, Jirka van der Roest, Remko van der Lugt, and Kees CJ Overbeeke. 2011. NOOT: a tool for sharing moments of reflection during creative meetings. In *Proceedings of the 8th ACM conference on Creativity and cognition*. 157–164.
- [17] Lisa A. Elkin, Matthew Kay, James J. Higgins, and Jacob O. Wobbrock. 2021. An Aligned Rank Transform Procedure for Multifactor Contrast Tests. In *The 34th Annual ACM Symposium on User Interface Software and Technology* (Virtual Event, USA) (UIST '21). Association for Computing Machinery, New York, NY, USA, 754–768. doi:10.1145/3472749.3474784
- [18] Corey Ford and Nick Bryan-Kinns. [n. d.]. On the Role of Reflection and Digital Tool Design for Creative Practitioners. ([n. d.]).
- [19] Corey Ford, Ashley Noel-Hirst, Sara Cardinale, Jackson Loth, Pedro Sarmento, Elizabeth Wilson, Lewis Wolstanholme, Kyle Worrall, and Nick Bryan-Kinns. 2024. Reflection Across AI-based Music Composition. In *Creativity and Cognition*. ACM, Chicago IL USA, 398–412. doi:10.1145/3635636.3656185 TLDR: This paper takes AI-based music composition as a case study to explore how artist-researcher composers reflected when integrating AIGC into their music composition process and found that many composers reflect on future directions in which to take their music whilst curating AIGC..
- [20] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. 2022. InCoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999* (2022).
- [21] Christina Guillaumier. 2016. Reflection as creative process: Perspectives, challenges and practice. *Arts and Humanities in Higher Education* 15, 3-4 (July 2016), 353–363. doi:10.1177/1474022216647381
- [22] Joshua Hailpern, Erik Hinterbichler, Caryn Leppert, Damon Cook, and Brian P Bailey. 2007. TEAM STORM: demonstrating an interaction model for working with multiple ideas during creative group work. In *Proceedings of the 6th ACM SIGCHI Conference on Creativity & Cognition*. 193–202.
- [23] Perttu Hämmäläinen, Mikke Tavast, and Anton Kunnari. 2023. Evaluating large language models in generating synthetic hci research data: a case study. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–19.
- [24] Nicolai Brodersen Hansen and Peter Dalsgaard. 2012. The productive role of material design artefacts in participatory design events. In *Proceedings of the 7th nordic conference on human-computer interaction: Making sense through design*. 665–674.
- [25] Caroline Hummels and Joep Frens. 2009. The reflective transformative design process. In *CHI'09 Extended Abstracts on Human Factors in Computing Systems*. 2655–2658.
- [26] Ellen Jiang, Edwin Toh, Alejandra Molina, Aaron Donsbach, Carrie J Cai, and Michael Terry. 2021. GenLine and GenForm: Two Tools for Interacting with Generative Language Models in a Code Editor. In *Adjunct Proceedings of the 34th Annual ACM Symposium on User Interface Software and Technology*. ACM, Virtual Event USA, 145–147. doi:10.1145/3474349.3480209 TLDR: A macro system with two tools that allow users to invoke language model prompts as macros in a code editor, and a form-like interface where the user provides input that is then transformed into multiple pieces of output at the same time..
- [27] Martin Jonsson and Jakob Tholander. 2022. Cracking the code: Co-coding with AI in creative programming education. In *Creativity and Cognition*. ACM, Venice Italy, 5–14. doi:10.1145/3527927.3532801 TLDR: A study of a group of university students using generative machine learning to translate from natural language to computer code explores how the use of the AI tool can be understood in terms of co-creation, focusing on how the tool may serve as a resource for understanding and learning..
- [28] Majeed Kazemitabaar, Runlong Ye, Xiaoning Wang, Austin Zachary Henley, Paul Denny, Michelle Craig, and Tovi Grossman. 2024. CodeAid: Evaluating a Classroom Deployment of an LLM-based Programming Assistant that Balances Student and Educator Needs. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. ACM, Honolulu HI USA, 1–20. doi:10.1145/3613904.3642773 TLDR: CodeAid, an LLM-powered programming assistant delivering helpful, technically correct responses, without revealing code solutions, is developed, revealing four design considerations for future educational AI assistants..
- [29] Hannmeng Liu, Ruoxi Ning, Zhiyang Teng, Jian Liu, Qiji Zhou, and Yue Zhang. 2023. Evaluating the logical reasoning ability of chatgpt and gpt-4. *arXiv preprint arXiv:2304.03439* (2023).
- [30] Lauren McCarthy, Casey Reas, and Ben Fry. 2015. *Getting started with P5.js: Making interactive graphics in JavaScript and processing*. Maker Media, Inc.
- [31] Andrew M McNutt, Anton Outkine, and Ravi Chugh. 2023. A Study of Editor Features in a Creative Coding Classroom. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. ACM, Hamburg Germany, 1–15. doi:10.1145/3544548.3580683 TLDR: An experimental editor is implemented and used to teach a sequence of college and high-school creative coding courses to identify opportunities to improve creativity- and novice-focused IDEs and highlight tensions in their design..
- [32] Philip Mendels, Joep Frens, and Kees Overbeeke. 2011. Freed: a system for creating multiple views of a digital collection during the design process. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 1481–1490.
- [33] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. A conversational paradigm for program synthesis. *arXiv e-prints* (2022), arXiv–2203.
- [34] Kylie Peppeler and Yasmin Kafai. 2005. Creative coding: Programming for personal expression. Retrieved August 30, 2008 (2005), 314.
- [35] Steven I Ross, Fernando Martinez, Stephanie Houde, Michael Muller, and Justin D Weisz. 2023. The programmer’s assistant: Conversational interaction with a large language model for software development. In *Proceedings of the 28th International Conference on Intelligent User Interfaces*. 491–514.

- [36] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950* (2023).
- [37] Kamran Sedig, Maria Klawe, and Marvin Westrom. 2001. Role of interface manipulation style and scaffolding on cognition and concept learning in learnware. *ACM Transactions on Computer-Human Interaction (TOCHI)* 8, 1 (2001), 34–59.
- [38] Phoebe Sengers, Kirsten Boehner, Shay David, and Joseph 'Jofish' Kaye. 2005. Reflective design. In *Proceedings of the 4th decennial conference on Critical computing: between sense and sensibility*. 49–58.
- [39] Moushumi Sharmin and Brian P Bailey. 2013. ReflectionSpace: an interactive visualization tool for supporting reflection-on-action in design. In *Proceedings of the 9th ACM Conference on Creativity & Cognition*. 83–92.
- [40] Sarah Stermann, Molly Jane Nicholas, Janaki Vivrekar, Jessica R Mindel, and Eric Paulos. 2023. Kaleidoscope: A Reflective Documentation Tool for a User Interface Design Course. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) (CHI '23). Association for Computing Machinery, New York, NY, USA, Article 702, 19 pages. doi:10.1145/3544548.3581255
- [41] Ben Swanson, Kory Mathewson, Ben Pietrzak, Sherol Chen, and Monica Dinulescu. 2021. Story centaur: Large language model few shot learning as a creative writing tool. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. 244–256.
- [42] Maike Vollstedt and Sebastian Rezat. 2019. An introduction to grounded theory with a special focus on axial coding and the coding paradigm. *Compendium for early career researchers in mathematics education* 13, 1 (2019), 81–100.
- [43] Anqi Wang, Jiahua Dong, Lik-Hang Lee, Jiachuan Shen, and Pan Hui. 2025. A Survey on Deep Learning for Design and Generation of Virtual Architecture. *Comput. Surveys* 57, 2 (Feb. 2025), 1–41. doi:10.1145/3688569 TLDR: This work identifies the principles of virtual architecture and illuminate its current production challenges, including datasets, multimodality, design intuition, and generative frameworks, and introduces the latest approaches to designing and generating virtual buildings leveraging 3D shape generation..
- [44] Jacob O. Wobbrock, Leah Findlater, Darren Gergle, and James J. Higgins. 2011. The aligned rank transform for nonparametric factorial analyses using only anova procedures. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Vancouver, BC, Canada) (CHI '11). Association for Computing Machinery, New York, NY, USA, 143–146. doi:10.1145/1978942.1978963
- [45] Daisy Yoo, Alina Hultgren, Jill Palzkill Woelfer, David G Hendry, and Batya Friedman. 2013. A value sensitive action-reflection model: evolving a co-design space with stakeholder and designer prompts. In *Proceedings of the SIGCHI conference on human factors in computing systems*. 419–428.
- [46] Zhongyi Zhou, Jing Jin, Vrushank Phadnis, Xiuxiu Yuan, Jun Jiang, Xun Qian, Kristen Wright, Mark Sherwood, Jason Mayes, Jingtao Zhou, Yiyi Huang, Zheng Xu, Yinda Zhang, Johnny Lee, Alex Olwal, David Kim, Ram Iyengar, Na Li, and Ruofei Du. 2025. InstructPipe: Generating Visual Blocks Pipelines with Human Instructions and LLMs. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. ACM, Yokohama Japan, 1–22. doi:10.1145/3706598.3713905 TLDR: Both technical and user evaluation shows that InstructPipe empowers users to streamline their ML pipeline workflow, reduce their learning curve, and leverage open-ended commands to spark innovative ideas..